CLI: https://www.postgresql.org/docs/current/reference-client.html

*** usage: DB connection**
Defaults are:

host	unix socket
port	5432
dbname	$(whoami)
user	$(whoami)
password	<none>

4 ways to specify connection:
1) libpq **connection string**
2) libpq **URI**
3) libpq **environment variables**
4) libpq service file
4) CLI options

See **libpq** below for details.

e.g.

```
# 1) libpq connection string
$ psql "dbname=mydb host=localhost port=5432 user=yui
password=pwd" ...


# 2) libpq URL
$ psql postgresql://yui:pwd@localhost:5432/mydb ...


# 3) libpq environment variable
$ PGHOST=localhost PGDATABASE=mydb PGPORT=5432
PGUSER=yui PGPASSWORD=pwd psql ...


# 4) libpq service file
$ PGSERVICEFILE=myservices.conf psql service=db ...
$ cat myservices.conf
[db]
host=localhost
port=5432
user=yui
password=pwd
dbname=mydb


# 5) CLI
$ psql --host=localhost \
  --port=5432 \
  --user=yui \
  ...
  mydb
```

*** CLI**
DB connection

```
## Common options
#
# Defaults
#  host=local socket
#  port=5432
#  user=same as $(whoami)
#  password=none
#  database=same as user
psql postgres                       # all default
psql --host= /tmp \                 # same
  --port=5432 \
  --username=$(whoami) \
  --no-password \
  postgres

psql --password ...                 # prompt password
psql --no-password ...              # no password

# connection string
psql "dbname=postgres host=/tmp port=5432 user=yui" ...
         # with connection string
psql  postgresql://yui:5432/postgres?host=/tmp" ...
         # with URL


#### General
# pg_isready: show connection info
pg_isready                              #
pg_isready -h /tmp -p 5432              # explicit


# pg_config: show config
pg_config
pg_config --binddir                     # show only
specific values


# User management
#
# Permissions:
#
#  createdb           can create DB. Default NO
#  createrole         can create roles i.e. users.
Default unspecified
#  login              can login. Default YES
#  replication        can initiate replacation. Default
unspecified
```

```
#  superuser          is superuser. Default NO
#
createuser -h localhosts -p 5432 ...
createuser --interactive yui                    #
createuser --role=yuinishizawa  --inherit...    # Member
of yuinishizawa
createuser --role=yuinishizawa  --no-inherit...#
createuser --pwprompt yui                       # Set
password
createuser --createdb \                         #
permissions
  --no-createdb \
  --login \
  --no-login \
  --pwprompt \
  --createrole \
  --no-createrole
  --superuser \
  --no-super-user \
  --replication \
  --no-replication \

dropuser yui
dropuser \
  --echo \
  --interactive \
  yui


## Database management
#
# create database
createdb mydb
createdb \
  --echo \
  --owner=yui \
  mydb

dropdb mydb
dropdb \
  --echo \
  --force \               # force close connects
  --interactive \
  --if-exists \           # no error even if database
doesn't exist
  mydb


# dump, restore
pg_dump
pg_restore


#
```

```
# pg_amcheck: check for corruption
pg_amcheck

# pg_bench: benchmark
pg_bench

# reindexdb: reindex Databases
reindexdb

# vacuumdb: garbage collect
vacuumdb


## cluster
clusterdb
pg_dumpall
pg_basebackup
pg_verifybackup


## misc
ecpg
pg_receivewal
pg_recvlogical
```

==========

== libpq ==

32: libpq: https://www.postgresql.org/docs/current/libpq.html

libpq
Core programmatic **API library** for PostgreSQL

*** Connection parameters: PostgreSQL URI, connection strings**
32.1.1 connection string: https://www.postgresql.org/docs/current/libpq-connect.html

connection string
PostgreSQL URI
A way to specify connection as a **single string**.
A connection string is a sequence of **key=value** pairs.

e.g.

```
# connection string
$ psql "dbname=mydb host=localhost port=5432 user=yui
password=pwd" ...

# URL
$ psql postgresql://yui:pwd@localhost:5432/mydb ...

# Same
```

```
psql postgresql://localhost/mydb?
port=5432&user=yui&password=pwd
```

URI
Format
#
postgresql://[user[:password]@][host][:port][/dbname][?params...]
#
postgresql://yui:pwd@localhost:5432/mybd fully specified
postgresql:// all default i.e.
host=socket port=5432 user=$(whoami) database=$(whoami)
postgresql://localhost
postgresql://localhost/mydb?port=5432&user=yui with parameters
postgresql://mydb?host=/tmpsock e.g. with socket

Parameters
#
host=localhost
port=5432
user=yui
password=abc
passfile=./pass
service=
sslmode=verify-full
... etc

*** Environment variables**
32.15 libpq variables: https://www.postgresql.org/docs/current/libpq-envars.html

e.g.

```
$ PGDATABASE=mydb PGUSER=yui PGPASSWORD=... psql
```

PSQLRC Default is ~/.psqlrc
SHELL
TMPDIR Default is /tmp

libpq variables
PGHOST host
PGPORT port
PGUSER username
PGPASSWORD
PGDATABASE

... etc ...

*** file: pgpass password file**
32.16: https://www.postgresql.org/docs/current/libpq-pgpass.html

pgpass
File for managing passwords

*** file: pg_service.conf services file**

32.17 service file: https://www.postgresql.org/docs/current/libpq-pgservice.html

pg_service.conf
File for managing service parameters

e.g.

```
# ~/.pg_service.conf
[mydb]
host=localhost
port=5432
user=yuinishizawa
dbname=mydb
```

Use

```
# Specify service
$ psql service=mydb

#
$ PGSERVICEFILE=~/.pg_service.conf psql service=mydb
```

==========

== psql ==

psql: https://www.postgresql.org/docs/current/app-psql.html

```
## Basics
#
$ psql                                      # Login to
db=$(whoami)
$ psql --host=localhost \                   # Same as
default
    --username="$(whoami)" \
    --no-password \
    --port=5432 \
    "$(whoami)"
$ psql mydb                                 # db=mydb
$ psql postgres                             # db=postgres
is created automatically initially
$ psql -h localhost -U "$(whoami)" -w ...  # Same

# script
$ psql -f script.sql ...
$ psql -f script.sql --no-pqslrc ...

# command
$ psql -c "SELECT 1" ...
$ psql -c "SELECT 1" --no-psqlrc ...

$ psql --list ...         # List databases
```

```
$ psql -l ...
```

*** Pattern**
man psql Patterns

Patterns are used in list commands
e.g.

```
\dt pg_catalog.*
```

*	any character
?	single character
[a-z]	range

mytable	Exact match
ytabl	BAD. Needs to be full match
ytabl	OK. Matches "mytable"

public.mytable	specify schema
mydb.public.mytable	specify db + schema
mydb.*.*	all schemas and tables
.	same
mydb.*	BAD. "mydb" is treated as a schema
..*	BAD. Can't reference other DBs

*** psqlrc**
Wiki: https://wiki.postgresql.org/wiki/Psqlrc
Example1: https://github.com/e7e6/psqlrc/blob/master/psqlrc

*** Variables (\set command)**
man psql: Variables section
help

\? variables	documentation
\set	list of current variables

Variables
Similar to Unix variables, set by **\set** command.
A **key-value** pair, where value is a **string**

NOTE: Use quotes :'a' to

```
> \set a 'a b c'

> select :a;                                BAD
ERROR:  syntax error at or near "c"
LINE 1: select a b c
                  ^

> select :'a';                              GOOD
?column? | a b c
```

e.g. can be used as a query

```
> \set count 'select count(*) from'
> :count mytable;                          select
count(*) from mytable
```

```
# Query options
AUTOCOMMIT              on          on|off
                                    Query not executed until explicit
COMMIT if off
ON_ERROR_STOP                       on|off
ON_ERROR_ROLLBACK                   on|off|interactive
SINGLELINE              off         on|off
                                    No need to terminate with ';' if off
SINGLESTEP              off         on|off
                                    Confirm each time before executing


# Last query info
ERROR                       false   true|false
SQLSTATE
LASTOID
ROW_COUNT                           number of rows affected
LAST_ERROR_MESSAGE
LAST_ERROR_SQLSTATE

# Last shell command info
SHELL_ERROR
SHELL_EXIT_CODE


# Output options
ECHO                        none    all|errors|none|queries
                                    Show SQL query executed
ECHO_HIDDEN                 off     on|off|noexec
                                    Show SQL query of
commands e.g. \d
FETCH_COUNT                 0       number of rows to fetch at a
time
HIDE_TABLEAM                off     on|off
                                    table access methods
HIDE_TOAST_COMPRESSION      off     on|off
                                    compression
SHOW_ALL_RESULTS        on          on|off
SHOW_CONTEXT            errors      never|errors|always


# Connection info
DBNAME
HOST
PORT
USER

# Server info
SERVER_VERSION_NAME
SERVER_VERSION_NUM
VERSION
```

VERSION_NAME
VERSION_NUM

psql options

COMP_KEYWORD_CASE		lower|upper|preserve-lower|preserve-upper
		auto-complete options
PROMPT1		
PROMPT2		
PROMPT3		
QUIET	off	on|off
VERBOSITY	default	
ENCODING		
IGNOREEOF	0	

History

HISTCONTROL	none	
HISTFILE		
HISTSIZE	500	

```
-- Variables
--
\? variables
\set                    -- see current values
\set a abc              -- a=abc
\set a 'abc'            -- same
\set a "abc"            -- BAD  a="abc"    double quotes
are literal
\set a a b c            -- BAD  a=abc      spaces are
removed
\set a 'a b c'          -- GOOD a=a b
\set a 'a\'b'           -- GOOD a=a'b      escape single
quote '


-- usage: variable with multiline string
-- ref: https://stackoverflow.com/questions/31454108/
defining-multi-line-strings-in-psql
> SELECT
$$
1
2
3
$$ AS a \gset
> echo :a

\unset a                -- unset
\getenv a HOME          -- set environment variable on
variable
                        -- e.g. a=$HOME

\echo :a                -- get
\echo :DBNAME           -- get special variable
```

```
\echo :'a'            -- with quotes i.e. as SQL string
\echo :"a"            -- with double quotes i.e. as SQL
name

SELECT :a;            -- use in query
SELECT :'a';          -- as string value
SELECT * FROM :"a";   -- as identifier (table name)
SELECT :"a" ...       -- as identifier (column name)

\! echo :'a'          -- ERROR can't use in command

\set q 'SELECT 1;'    -- as SQL
:a
```

*** Variable: PROMPT**

See man psql

%%	'%' escaped
%M	Host [local] unix socket 1.2.3.4 URL
%m	short host truncated at first . e.g. db.1.com -> db
%>	port
%n	user name
%/	DB name
%~	DB name, or ~ if default DB
%#	Prompt # super user > normal user
%x	transaction state <empty> * in transaction ! failed transaction ? unknown e.g. no connection

%

```
# ANSI color
%[%033[31m%]          \033[31m  31=red foreground
%[%033[41m%]          \033[42m  42=green background
%[%033[31;42m%]       \033[31;42m   foreground+background
%[%033[om%]           \033[0m     0=reset
```

*** Printing options (\pset)**

See man psql or \? variables

	Default	**No value**
columns	0	column width 0=auto
footer	on	(n rows) footer

null	'' (empty)			
numericlocale	off			
pager				
pager_min_lines				
# Formatting				
expanded	off	toggle	show data as individual records	
tuples_only	off	toggle	show data onxly i.e. no headers, etc	
title	''	reset	table title	
format	aligned		aligned unaligned	default minimal
formatting with **fieldsep**				columns rows with
recordsep				
			wrapped html csv latex, latext-longtable troff-ms	
border	1		number of borders	

```
-- format=aligned (default)
> \dS
                            List of relations
┌──────────────┬──────────────────────────────────────┬──────────
──┬─────────────────┐
│   Schema     │              Name                    │   Type
│    Owner     │
├──────────────┼──────────────────────────────────────┼──────────
──┼─────────────────┤
| pg_catalog | pg_aggregate                         | table
| yuinishizawa |
| pg_catalog | pg_am                                | table
| yuinishizawa |
...


-- format=unaligned
-- fieldsep='|'
-- recordsep='\n'
> \dS
List of relations
Schema|Name|Type|Owner
pg_catalog|pg_aggregate|table|yuinishizawa
pg_catalog|pg_am|table|yuinishizawa
pg_catalog|pg_amop|table|yuinishizawa
...
```

```
-- format=html
> \dS
<table border="10">
  <caption>List of relations</caption>
  <tr>
    <th align="center">Schema</th>
    <th align="center">Name</th>
    <th align="center">Type</th>
    <th align="center">Owner</th>
  </tr>
  <tr valign="top">
    <td align="left">pg_catalog</td>
    <td align="left">pg_aggregate</td>
    <td align="left">table</td>
    <td align="left">yuinishizawa</td>
  </tr>
 ...

-- format=csv
> \dS
Schema,Name,Type,Owner
pg_catalog,pg_aggregate,table,yuinishizawa
pg_catalog,pg_am,table,yuinishizawa
pg_catalog,pg_amop,table,yuinishizawa
...


-- tuples_only=on
> \dS
| pg_catalog | pg_aggregate                    | table
| yuinishizawa |
| pg_catalog | pg_am                           | table
| yuinishizawa |
...

-- expanded=on
> \dS
List of relations
[ RECORD 1 ]-----------------------------
| Schema | pg_catalog                      |
| Name   | pg_aggregate                    |
| Type   | table                           |
| Owner  | yuinishizawa                    |
[ RECORD 2 ]-----------------------------
| Schema | pg_catalog                      |
| Name   | pg_am                           |
| Type   | table                           |
| Owner  | yuinishizawa                    |
[ RECORD 3 ]-----------------------------
| Schema | pg_catalog                      |
..
```

csv_fieldsep	','	
tableattr	'' (empty)	HTML: <table> tag attribute
fieldsep	'|'	separator in unaligned format
fieldsep_zero		
recordsep	'\n'	separator in unaligned format
recordsep_zero	off	

linestyle	ascii	acsii, old-ascii, unicode
unicode_border_linestyle	single	
unicode_column_linestyle	single	
unicode_header_linestyle	single	

```
-- ascii
> \dS
                          List of relations
   Schema   |              Name               |   Type
|    Owner
------------+---------------------------------
+----------+--------------
 pg_catalog | pg_aggregate                    | table
| yuinishizawa
 pg_catalog | pg_am                           | table
| yuinishizawa
...

-- unicode + unicode_border
> \dS
                          List of relations
   Schema   ‖              Name               ‖   Type
‖    Owner
═══════════╬═══════════════════════════════════╬═══════
═╬═══════════════
 pg_catalog ‖ pg_aggregate                    ‖ table
‖ yuinishizawa
 pg_catalog ‖ pg_am                           ‖ table
‖ yuinishizawa
...
```

```
-- \pset: printing options
\? variables          -- help on available options
\pset                 -- show all current values
\pset border 1        -- set

-- no value: depends on option
-- May show current value, toggle, or unset
\pset border          -- show current value
\pset title           -- unset title
\pset tuples_only     -- toggle on
```

```
\pset tuples_only     -- off
```

*** Query buffer**

Query buffer
A buffer of command/queries.
Either the **current unexecuted**, or the **last executed**.
In particular, useful for using an **editor** to write the commands.

usage: workspace

Step 1: open workspace

```
-- Open editor
> \e workspace.sql
```

In vim

```
select 1
 union select 2;
```

Step 2: execute on edit

```
> \e workspace.sql

 ?column?
══════════
        1
        2
(1 row)

-- Reopen
> \e workspace.sql
```

```
-- Commands
\e                    -- edit buffer in EDITOR
\e mybuffer.sql       -- load file "mybuffer" into buffer
and open in EDITOR
\ef                   -- edit function
\w mybuffer.sql       -- save current buffer in file
"mybuffer"

\p                    -- print current buffer
\r                    -- reset buffer
\s                    -- show history
\s psqlhistory        -- save history to file
```

*** Descriptions (\d* commands)**

```
---- Description commands
--
-- common
\dt                   -- list all tables in default
schema (public by default)
\dt mytable           -- list mytable
\dt public.*          -- pattern
\dt+ *                -- show more info
\dtS *                -- show system


-- \d: describe various objects in schema
-- tables, views, sequences
\d                    -- describe current schema
\d mytable            -- describe table
\d myseq              -- describe sequence
\d mytype             -- describe custom composite type
```

```
-- Describe schema
mydb# \d

                List of relations
 Schema |      Name        |   Type   |    Owner
════════╪══════════════════╪══════════╪═══════════════
 public | myseq            | sequence | yuinishizawa
 public | yuitest          | table    | yuinishizawa
 public | yuitest_id_seq   | sequence | yuinishizawa
(10 rows)


-- Describe sequence
mydb# \d+ myseq
                     Sequence "public.myseq"
  Type   | Start | Minimum | Maximum | Increment |
Cycles? | Cache
═════════╪═══════╪═════════╪═════════╪═══════════╪═══════
═══╪═══════
 integer |     1 |       1 |       8 |         3 | no
|     1


-- Describe table
mydb# \d+ yuitest

Table "public.yuitest"
 Column |       Type        | Collation | Nullable |
Default               | Storage  | Compression | Stats
target |  Description
════════╪═══════════════════╪═══════════╪══════════╪═══
═══════════════════════════════╪══════════╪═════════════
═══╪═══════════════╪═══════════════
 id     | integer           |           | not null |
nextval('yuitest_id_seq'::regclass) | plain    |
|              | This is an ID
```

```
 id2    | integer           |           | not null |
generated always as identity        | plain    |
|              |
 sum    | integer           |           |          |
generated always as (x + y) stored  | plain    |
|              |
 label  | character varying |           | not null |
''::text                            | extended |
|              |
 x      | integer           |           |          |
| plain    |             |              |
 y      | integer           |           |          |
| plain    |             |              |
 t2id   | integer           |           |          |
| plain    |             |              |
 t      | integer           |           | not null |
| plain    |             |              |
Indexes:
    "yuitest_pkey" PRIMARY KEY, btree (id)
Check constraints:
    "b" CHECK (x > y)
    "c" CHECK (y > '-10'::integer)
Foreign-key constraints:
    "yuitest_t2id_fkey" FOREIGN KEY (t2id) REFERENCES
table1(id)
Access method: heap
```

```
---- Objects
\l                    -- databases

\dn                   -- schemas (n=namespaces)
\db                   -- tablespaces
\dt                   -- tables
\dP                   -- [only index/table] partitioned
relations [n=nested]
\dv                   -- views
\ds                   -- sequences
\dm                   -- materialized views
\dy                   -- event triggers
\dA                   -- access methods
\dX                   -- extended statistics

\dd                   -- show object descriptions not
displayed elsewhere
\sv  VIEWNAME         -- show a view's definition
\dl                   -- large objects, same as \lo_list


---- Data types
\dT               -- data types
\dT mylabel       -- enums
\d mytype         -- composite type
```

```
\dD              -- domains
\dc              -- conversions
\dC              -- casts
\dO              -- collations


---- Functions
\da                  -- aggregate function
\df                  -- function
\do                  -- operators e.g. + - *
\dfS+ array_*        -- e.g. array functions
\dL                  -- function languages
\sf cardinality                           -- function
definition
\sf array_fill(anyelement, integer[])     -- function
definition with argument
```

```
> \dfS+ array_*

    List of functions   | internal
-[ RECORD
1 ]-----+-----------------------------------------------
---------------------------
Schema              | pg_catalogcombine function
Name                |
array_agg-----------------------------------------------
--------------
Result data type    | anyarrayog
Argument data types | anyarrayg_array_deserialize
Type                | aggernal
Volatility          | immutableternal
Parallel            | safe
Owner               | yuinishizawa
Security            | invoker
Access privileges   | NULL
Language            | internal
Internal name       | aggregate_dummy
Description         | concatenate aggregate input into
an array
-[ RECORD
2 ]-----+-----------------------------------------------
---------------------------
Schema              | pg_catalog
Name                | array_agg
Result data type    | anyarray
Argument data types | anynonarray
Type                | agg
Volatility          | immutable
Parallel            | safe
Owner               | yuinishizawa
Security            | invoker
Access privileges   | NULL
Language            | internal
Internal name       | aggregate_dummy
```

```
Description         | concatenate aggregate input into
an array
...


> \sf+ array_fill(anyelement, integer[])
        CREATE OR REPLACE FUNCTION
pg_catalog.array_fill(anyelement, integer[])
         RETURNS anyarray
         LANGUAGE internal
         IMMUTABLE PARALLEL SAFE
1        AS $function$array_fill$function$
```

```
---- Roles
\dg                   -- roles (g=groups)
\du                   -- same as dg (u=users)
\drg                  -- role GRANTS
                      -- e.g. membership
```

```
> \drgS+
                        List of role grants
 Role name  |      Member of       |   Options    |
Grantor
------------+----------------------+--------------+-----
--------
 pg_monitor | pg_read_all_settings | INHERIT, SET |
yuinishizawa
 pg_monitor | pg_read_all_stats    | INHERIT, SET |
yuinishizawa
 pg_monitor | pg_stat_scan_tables  | INHERIT, SET |
yuinishizawa
 tempuser   | yuinishizawa         | INHERIT, SET |
yuinishizawa
(4 rows)

> \dg+
                                      List of roles
  Role name   |                          Attributes
| Description
--------------+-------------------------------------------
--------------------+-------------
 tempuser     | Cannot login
|
 yuinishizawa | Superuser, Create role, Create DB,
Replication, Bypass RLS |
```

```
---- Access privileges
--
--  a=access
--  r=read
--  w=write
--  D=truncate
--  x=execute
--  t=
--  m=
\dp                  -- table, view, and sequence access
privileges
\ddp                 -- default access privileges
```

```
> \dpS+

Access privileges
   Schema   |              Name               |   Type
|         Access privileges          |  Column
privileges   | Policies
------------+-------------------------------------+-------
-+-------------------------------------+-----------------
-----+-----------
 pg_catalog | pg_aggregate                        | table
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                     |
|
 pg_catalog | pg_am                               | table
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                     |
|
 pg_catalog | pg_amop                             | table
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                     |
|
 pg_catalog | pg_amproc                           | table
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                     |
|
 pg_catalog | pg_attrdef                          | table
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                     |
|
 pg_catalog | pg_attribute                        | table
```

```
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                    |
|
 pg_catalog | pg_auth_members                    | table
| yuinishizawa=arwdDxtm/yuinishizawa↵|
|
            |                                     |
| =r/yuinishizawa                    |
|
 pg_catalog | pg_authid                          | table
| yuinishizawa=arwdDxtm/yuinishizawa |
|
...
```

```
-- Configuration
\drds                -- role/database configuration
(rds=role/database settings)
\dconfig             -- server configuration i.e.
postgresql.conf
```

```
> \drds
                  List of settings
     Role     | Database |         Settings
--------------+----------+---------------------------
 yuinishizawa | NULL     | transaction_timeout=33s
 NULL         | mydb     | transaction_read_only=off
(2 rows)

> \dconfig+
                                      List of non-
default configuration parameters
         Parameter           |                        Value
|  Type   |  Context   | Access privileges
-----------------------------+-------------------------------
----------------------+---------+------------+----------
---------
 IntervalStyle               | postgres_verbose
| enum    | user       | NULL
 TimeZone                    | Asia/Tokyo
| string  | user       | NULL
 application_name            | psql
| string  | user       | NULL
 client_encoding             | UTF8
| string  | user       | NULL
 client_min_messages         | log
| enum    | user       | NULL
 config_file                 | /opt/homebrew/var/
postgresql@17/postgresql.conf | string  | postmaster |
NULL
```

```
 data_directory             | /opt/homebrew/var/
postgresql@17                | string  | postmaster |
NULL
 default_text_search_config | pg_catalog.english
| string  | user       | NULL
 hba_file                   | /opt/homebrew/var/
postgresql@17/pg_hba.conf     | string  | postmaster |
NULL
 ident_file                 | /opt/homebrew/var/
postgresql@17/pg_ident.conf   | string  | postmaster |
NULL
 lc_messages                | C
| string  | superuser  | NULL
 log_timezone               | Asia/Tokyo
| string  | sighup     | NULL
 transaction_timeout        | 10min
| integer | user       | NULL
 work_mem                   | 5MB
| integer | user       | NULL
```

```
---- Index info
\di               -- indexes
\dA               -- index types e.g. btree, gist
\dAc              -- operator classes (c=class)
\dAf              -- operator families (f=family)
\dAo              -- supported operations (o=operation)
e.g. < = >

\Adp              -- association functions (p=procedure?)

-- \dAc, \dAf options
\dAc+ btree               -- btree only
\dAc+ btree numeric       -- btree + classes for numeric
type only
\dAc+ * numeric           -- classes for numeric type
only
\dAf+ btree numeric

-- \dAo, \dAp options
\dAo+
\dAo+ btree
\dAo+ btree numeric_ops
\dAo+ * numeric_ops
\dAp+ btree numeric_ops
```

```
# All index types
> \dA+
                            List of access methods
  Name  | Type  |       Handler        |
```

```
Description
═══════╪═══════╪═══════════════════════╪═══════════════════════════════
═══════════════════════
 brin   | Index | brinhandler           | block range
index (BRIN) access method
 btree  | Index | bthandler             | b-tree index
access method
 gin    | Index | ginhandler            | GIN index
access method
 gist   | Index | gisthandler           | GiST index
access method
 hash   | Index | hashhandler           | hash index
access method
 heap   | Table | heap_tableam_handler | heap table
access method
 spgist | Index | spghandler            | SP-GiST index
access method


# All operator classes of btree
> \dAc+ btree
                                                 List of
operator classes
  AM   |          Input type           | Storage type |
Operator class    | Default? |  Operator family   |
Owner
═══════╪═══════════════════════════════╪══════════════╪═══
═══════════════════╪══════════╪═══════════════════════╪═══
═══════
 btree | "char"                        |              |
char_ops            | yes      | char_ops           |
yuinishizawa
 btree | anyarray                      |              |
array_ops           | yes      | array_ops          |
yuinishizawa
 btree | anyenum                       |              |
enum_ops            | yes      | enum_ops           |
yuinishizawa
 btree | anymultirange                 |              |
multirange_ops      | yes      | multirange_ops     |
yuinishizawa
 btree | anyrange                      |              |
range_ops           | yes      | range_ops          |
yuinishizawa
 btree | bigint                        |              |
int8_ops            | yes      | integer_ops        |
yuinishizawa
 btree | bit                           |              |
bit_ops             | yes      | bit_ops            |
yuinishizawa
 btree | bit varying                   |              |
varbit_ops          | yes      | varbit_ops         |
yuinishizawa
 btree | boolean                       |              |
bool_ops            | yes      | bool_ops           |
```

```
yuinishizawa
 ...

# All operator families of btree
> \dAf+ btree
                                        List of operator
families
  AM   |  Operator family   |
Applicable types                        |    Owner
═══════╪════════════════════╪═══════════════════════════════════════
═══════════════════════════════════════╪═══════════════
 btree | array_ops          | anyarray
| yuinishizawa
 btree | bit_ops            | bit
| yuinishizawa
 btree | bool_ops           | boolean
| yuinishizawa
 btree | bpchar_ops         | character
| yuinishizawa
 btree | bpchar_pattern_ops | character
| yuinishizawa
 btree | bytea_ops          | bytea
| yuinishizawa
 btree | char_ops           | "char"
| yuinishizawa
 btree | datetime_ops       | date, timestamp with time
zone, timestamp without time zone | yuinishizawa
 btree | enum_ops           | anyenum
| yuinishizawa
 ...


# All numeric operators supported by btree
> \dAo+ btree numeric_ops
                        List of operators of operator
families
  AM   | Operator family |      Operator       |
Strategy | Purpose | Sort opfamily
═══════╪═════════════════╪═════════════════════╪═══════
═══╪═══════╪═══════════════
 btree | numeric_ops     | <(numeric,numeric)  |
1 | search  |
 btree | numeric_ops     | <=(numeric,numeric) |
2 | search  |
 btree | numeric_ops     | =(numeric,numeric)  |
3 | search  |
 btree | numeric_ops     | >=(numeric,numeric) |
4 | search  |
 btree | numeric_ops     | >(numeric,numeric)  |
5 | search  |
(5 rows)


# All numeric support functions associated with btree
> \dAp+ btree numeric_ops
```

```
                                              List of
support functions of operator families
  AM   | Operator family | Registered left type |
Registered right type | Number |
Function
═══════╪═════════════════╪══════════════════════╪═══════
═══════════════════╪════════╪═══════════════════════════════════
═══════════════════
 btree | numeric_ops     | numeric              |
numeric               |      1 |
numeric_cmp(numeric,numeric)
 btree | numeric_ops     | numeric              |
numeric               |      2 |
numeric_sortsupport(internal)
 btree | numeric_ops     | numeric              |
numeric               |      3 |
in_range(numeric,numeric,numeric,boolean,boolean)
(3 rows)
```

```
---- Text
\dF               -- text search configurations
\dFd              -- text search dictionaries
\dFp              -- text search parsers
\dFt              -- text search templates

---- Replication
\dRp              -- replication publications
\dRs              -- replication subscriptions

---- Foreign
\dE               -- foreign tables
\des              -- foreign servers
\det              -- foreign tables
\dew              -- foreign-data wrappers
\deu              -- user mappings

---- Misc
\dx               -- extensions
```

*** if/else**

e.g. check database

```
SELECT :'DBNAME' = 'mydb' as is_mydb \gset
\if :is_mydb
 ....
\else
  \echo 'Not mydb'
  \q
\endif
```

```
-- note: boolean expression is the same as configuration
-- i.e.
--   true on yes 1        TRUE
--   false off no 0       FALSE
\if true              -- true|
\if on                -- OK  on|off
\if 1                 -- OK  1|0
\if 1                 -- OK  yes|no
\if :x                -- from variable
\if 'a'               -- ERROR Not a boolean.
\if 'a' = 'b'         -- ERROR Cannot compare here
  \echo 'a'
\elif on
\else
  \echo 'b'
\endif
```

*** Commands**

```
-- Comment

-- Help
\?                    -- Show psql help
\? commands           -- same
```

```
# See 'man psql' for details on each command
# e.g. \p command

\p or \print
  Print the current query buffer to the standard output.
If the current query buffer is empty, the most recently
executed query is printed instead.
```

```
\? options            -- CLI options
                      -- i.e. same as `psql --help`
\? variables          -- CLI variables
                      -- i.e. \set, \pset, \setenv
variables
\h                    -- List SQL commands
\h CREATE ROLE        -- SQL command help
\q                    -- Quit


-- Connection
\conninfo               -- Show current connection info
\c myDB                 -- Use database 'myDB'
\c myDB yui localhost   -- Use database 'myDB' as
yui@localhost


-- IO
\echo hello           -- print to stdout
```

```
\echo :a              -- e.g. variable if set
\echo -n ...          -- no new line
\warn ...             -- print to stderr
\qecho ...            -- print to \o output

\o out.txt            -- write SQL outputs to file
\o                    -- reset to default i.e. stdout
\i a.sql              -- read commands from file
\copy ...             -- COPY SQL query


-- Query & \g* commands
--
-- \g ...         execute query, with printing options
-- \gdesc         describe query only
--                i.e. equivalent to \d <result table
definition>
-- \bind          use query params
-- \gset          set query result on variables
-- \gexec         execute text as SQL
--
SELECT 1;
SELECT 1 \g                        -- same

-- \g [(option=value ...)] <file> | |<command>
SELECT ... \g out.txt                             -- pipe
to file
SELECT ... \g | grep hello                        -- pipe
to shell command
SELECT ... \g | grep abc | grep def             --
multiple
SELECT ... \g (format=html title=a) ...         -- set
printing i.e. \pset options
SELECT ... \g (format=html title=a) out.txt     -- e.g.
pipe with options

-- \gdesc
SELECT 1 \gdesc

-- \bind
SELECT $1, $2 \bind 1, 'a' \g      -- use parameters
\set q 'SELECT $1, $2'             -- e.g. with variable
q
:q \bind 1, 'a' \g

-- \gset:
SELECT 1 as a, 'B' as b \gset
\echo :a, :b                       -- e.g. 1, B
SHOW search_path \gset             -- OK. Works for SHOW
\echo :search_path                 -- '$user','public

-- \gexec
SELECT 'CREATE TABLE test ()' \gexec
```

```
-- \gdesc
> select * from yuitest \gdesc
┌────────┬────────────────────┐
| Column |       Type         |
├────────┼────────────────────┤
| id     | integer            |
| id2    | integer            |
| sum    | integer            |
| label  | character varying |
| x      | integer            |
| y      | integer            |
```

```
---- query
--
SELECT * FROM pg_catalog      -- ERROR
SELECT * FROM pg_catalog;
SHOW config_file;         -- config location e.g. /var/
lib/postgresql/data/postgresql.conf
```

==========

======

==admin==

*** pg_dump/restore**

```
## pg_dump
pg_dump
  # general options
  -v
  --verbose
  -V
  --version
  -?
  --help


  # Output
  -f file
  --file=mydb.dump
  -F p
  --format=p          # plain text (default). Restore
with psql instead of pg_restore
  --format=c          # custom format
  --format=d          # directories
  --format=t          # tar

  -E encoding
  --encoding=encoding
  -Z level
  -Z method[:detail]
```

```
  --compress=level
  --compress=method[:detail]


  # Data
  --large-objects
  -b
  --no-large-objects
  -B


  # Filters
  -n pattern
  --schema=pattern

  -N pattern
  --exclude-schema=pattern

  -t pattern
  --table=pattern

  -T pattern
  --exclude-table=pattern
  -e pattern
  --extension=pattern

  --table-and-children=pattern

  --section=sectionname

  --disable-triggers
  -S username
  --superuser=username

  --exclude-extension=pattern
  --exclude-table-and-children=pattern
  --exclude-table-data=pattern
  --exclude-table-data-and-children=pattern

  --filter=filename
```

```
# include file format
include table mytable*
exlucde table ...
include schema ...
include function ...
include index ...
include trigger ...
```

```
  # DB meta-data to exclude
  -O
  --no-owner                      # no ALTER OWNER
```

```
  -R
  --no-privileges                 # no GRANT PRIVILEGE
  --no-acl
  --no-comments
  --no-publications
  --no-security-labels
  --no-subscriptions
  --no-table-access-method
  --no-tablespaces
  --no-toast-compression
  --no-unlogged-table-data


  # Options
  --create                    # include CREATE DATABASE
  -C                          # If --clean is specified,
DROP DATABASE first

  -c                          # DROP <object> first for
each dumped object e.g. DROP TABLE
  --clean                     # Only allowed if not
data-only
  --data-only                 # Only dump data e.g. no
CREATE TABLE
  -a
  --schema-only
  -s

  --create                    # case: database does not
exist yet
  --create --clean            # case: re-create whole
database
  --clean                     # case: use existing
database, but re-create each dumped table, index, view,
etc
  --data-only                 # case: use existing
database + existing tables, views, etc
  --data-only --clean         # ERROR Can't clean if
data-only since the table cannot be created
```

```
# e.g.
#
# Default: does NOT remove DB or table
# --format=plain
CREATE TABLE public.t1 (
    x integer,
    y integer,
    sum integer GENERATED ALWAYS AS ((x + y)) STORED
);
COPY public.t1 (x, y) FROM stdin;
0   0
...
CREATE INDEX t1_x_y_idx ON sch_slp.t1 USING btree (x,
```

```
y);


# case: DROP TABLE to reset table
# --format=plain --clean
DROP INDEX public.t1_x_y_idx;
DROP TABLE public.t1;
CREATE TABLE public.t1 (
    x integer,
    y integer,
    sum integer GENERATED ALWAYS AS ((x + y)) STORED
);
COPY public.t1 (x, y) FROM stdin;
0   0
...


# case: CREATE DATABASE to initialize database
# --format=plain --create
CREATE DATABASE mydb WITH TEMPLATE = template0 ENCODING
= 'UTF8' LOCALE_PROVIDER = libc LOCALE = 'en_US.utf8';
ALTER DATABASE mydb OWNER TO admin;
\connect mydb
CREATE TABLE public.t1 (
    x integer,
    y integer,
    sum integer GENERATED ALWAYS AS ((x + y)) STORED
);
COPY public.t1 (x, y) FROM stdin;
0   0
...

# case: DROP DATABASE to reset whole database
# --format=plain --create --clean
DROP DATABASE mydb;
CREATE DATABASE mydb WITH TEMPLATE = template0 ENCODING
= 'UTF8' LOCALE_PROVIDER = libc LOCALE = 'en_US.utf8';
ALTER DATABASE mydb OWNER TO admin;
\connect mydb
CREATE TABLE public.t1 (
    x integer,
    y integer,
    sum integer GENERATED ALWAYS AS ((x + y)) STORED
);
COPY public.t1 (x, y) FROM stdin;
0   0
...


# case: data only
# --format=plain  --data-only
COPY public.t1 (x, y) FROM stdin;
0   0
...
```

```
  --strict-names
  --if-exists                 # include IF EXISTS
  --quote-all-identifiers
  --disable-dollar-quoting

  --inserts                       # output data as
INSERT
  --column-inserts
  --attribute-inserts
  --rows-per-insert=nrows

  --extra-float-digits=ndigits
  --use-set-session-authorization



  # Running
  -j njobs
  --jobs=njobs                    # parallel. Only for
format=directory
  --load-via-partition-root
  --lock-wait-timeout=timeout
  --on-conflict-do-nothing
  --serializable-deferrable
  --snapshot=snapshotname
  --no-sync
  --sync-method=method




## pg_restore
pg_restore -d mydb ./mydb.dump
pg_restore
  -?
  --help

  -v
  --verbose
  -V
  --version

  # Basic options
  -d dbname
  --dbname=dbname
  -e
  --exit-on-error
```

```
  # Output
  -f filename                 # ?
  --file=filename

  # Data: same as dump
  -s
  --schema-only
  -a
  --data-only

  # Filters: same as dump
  -n schema
  --schema=schema
  -N schema
  --exclude-schema=schema
  -t table
  --table=table
  -I index
  --index=index
  -P function-name(argtype [, ...])
  --function=function-name(argtype [, ...])
  -T trigger
  --trigger=trigger
  --section=sectionname
  -S username
  --superuser=username
  --filter=filename


  -l                          # list files in the dump
  --list
  -L list-file
  --use-list=list-file

  #
  -O
  --no-owner
  -x
  --no-privileges
  --no-acl
  --no-comments
  --no-data-for-failed-tables
  --no-publications
  --no-security-labels
  --no-subscriptions
  --no-table-access-method
  --no-tablespaces


  # NOTE: clean/create only works
  # if the dump contains the schema
  # i.e. does NOT work for --data-only dumps
  -c                                  # DROP object
(e.g. table) before restoring. Implies --create?
```

```
  --clean
  -C                                  # CREATE object
before restoring.
  --create

  # Options
  --enable-row-security
  --if-exists
  --strict-names
  --use-set-session-authorization


  # Runtime
  -j number-of-jobs                   # parallel. Only
for format=directory or custom
  --jobs=number-of-jobs
  -1
  --single-transaction
  --disable-triggers
  --transaction-size=N
```

*** Example: basic dump/restore**
Dump format needs to be non-text

```
# Dump
$ pg_dump \
   -v \
   --file=mydb.dump \
   --format=d \
   -h localhost -U yui mydb

# Restore
# Note: --dbname is the database used while restoring
$  pg_restore \
  -v --exit-on-error \
  --dbname=postgres \
  -h localhost -U yui mydb.dump
```

*** Example: dump/restore with plain text i.e. restore with psql**
Use **psql** to restore

```
# Dump
$ pg_dump \
   -v \
   --file=mydb.dump \
   --format=p \
   -h localhost -U yui mydb

# Restore
$ psql -h localhost postgres < mydb.dump
```

*** Example: show dump content + restore only selected content**
Use -l and -L options

Step 1) Output **object list**

```
$ pg_restore -l mydb.dump > db.list
```

Step 2) Edit db.list file

```
;
; Archive created at 2025-03-24 05:33:37 JST
;     dbname: slp
;     TOC Entries: 177
;     Compression: gzip
;     Dump Version: 1.16-0
;     Format: DIRECTORY
;     Integer: 4 bytes
;     Offset: 8 bytes
;     Dumped from database version: 14.6
;     Dumped by pg_dump version: 17.4 (Homebrew)
;
;
; Selected TOC Entries:
;
649; 1259 1760386 TABLE sch_slp m_object_type
yuinishizawa
716; 1259 1761029 TABLE sch_slp t_blank_map_layout
yuinishizawa
767; 1259 1761406 TABLE sch_slp
t_blank_map_layout_202003 yuinishizawa
768; 1259 1761414 TABLE sch_slp
t_blank_map_layout_202004 yuinishizawa
...
```

Step 3) Import from list

```
$ pg_restore -L db.list -v -d mydb mydb.dump
```

==========
======

```
This
is
code
Monaco 14
```